



第03章 （流程控制之）分支语句

张仁斌@合肥工业大学软件学院

本章主要内容



3.1 **bool**（布尔）数据类型、关系运算符与布尔表达式



3.2 **if**语句和**if-else**语句

3.3 **if**语句常见错误和陷阱

3.4 逻辑运算符

3.5 **switch**语句

3.6 条件表达式

3.7 运算符优先级和结合律

3.8 作业与实践

引例



- ◆ 在计算圆的面积时，若输入的半径radius是负数，程序不应计算面积，而是执行提示或其它操作

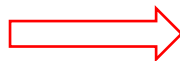
```
#include <iostream>
using namespace std;

int main()
{
    double radius, area;
    const double PI = 3.14159;
    cout << "Enter a radius: ";
    cin >> radius; // 若输入值无效，如何处理？

    area = radius * radius * PI;

    cout << "The area is " << area << endl;

    return 0;
}
```



```
#include <iostream>
using namespace std;
int main() {
    double radius, area;
    const double PI = 3.14159;
    cout << "Enter a radius: ";
    cin >> radius;
    if (radius < 0)
    {
        cout << "Incorrect Input" << endl;
    }
    else
    {
        area = radius * radius * PI;
        cout << "The area is " << area << endl;
    }
    return 0;
}
```

布尔（bool）表达式，值为布尔值true或false的表达式

bool数据类型与关系运算符



◆ **bool**数据类型用于定义一个**布尔变量**，可以赋值**布尔值true或false**

■ 例如：bool a = false;

◆ **C/C++**提供了**6种关系运算符**用于比较两个值的大小

■ 注意“等于”运算符是两个等号（==），一个等号（=）是赋值运算符

■ 关系运算符的结果是布尔值true或false（跟整数文字“10”一样，true和false是布尔文字，是不能作为标识符的关键字）

操作符	名称	示例
>	大于	a > b
>=	大于或等于	a >= b
<	小于	2 < 1
<=	小于或等于	c <= d
==	等于（恒等于）	1 == c
!=	不等于	1 != 3

```
int a = 10;
int b = 9;
a == b+1    运算结果: true;
a == b      运算结果: false;
a > b       运算结果: true;
a >= b      运算结果: true;
b > a       运算结果: false;
a >= b+1    运算结果: true;
a <= b+1    运算结果: true;
a != b      运算结果: true;
```



◆在C/C++内部，**1代表true，0代表false**，在控制台显示一个布尔值时，**true显示为1，false显示为0**

```
cout << (4 >= 5); // 显示0
```

◆与布尔值相关的数据类型转换

■在C/C++中，可以把任意一个数值赋给一个布尔变量，任意**非0值都是true**，0是false，例如：

```
bool a = -1.5; // a为true
```

```
bool b = 0;    // b为false
```

```
bool c = true;
```

```
int k = c;     // k为1
```

本章主要内容



3.1 bool（布尔）数据类型、关系运算符与布尔表达式

3.2 if语句和if-else语句



3.3 if语句常见错误和陷阱

3.4 逻辑运算符

3.5 switch语句

3.6 条件表达式

3.7 运算符优先级和结合律

3.8 作业与实践



三种程序流程结构

◆ 顺序结构

程序总体上都是顺序结构

◆ 分支结构（也称作选择结构、分支选择结构）

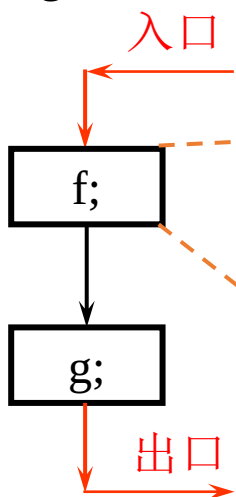
◆ 循环结构

顺序结构

算法描述：

f; //语句(块)

g;



分支结构

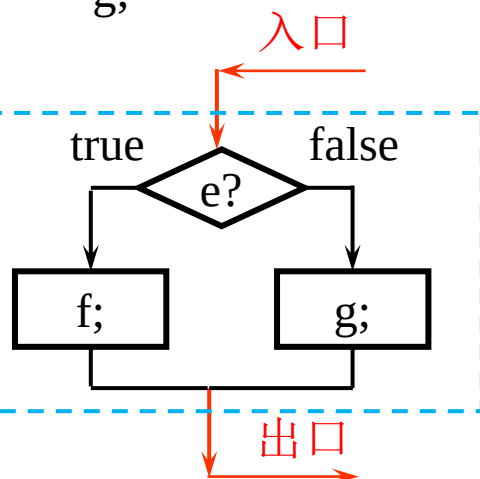
算法描述：

if(booleanExpression)

f;

else

g;

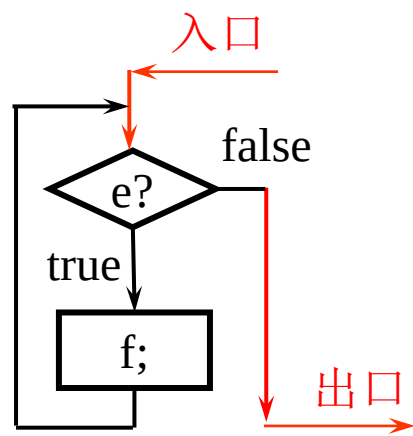


循环结构（后续章节讲授）

当型循环

while(booleanExpression)

f; // 循环体

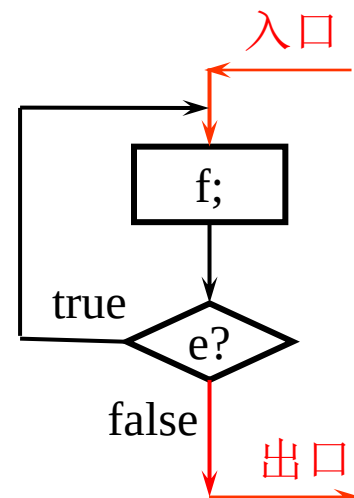


直到循环

do

f ; // 循环体

while(booleanExpression)



if语句



◆ C/C++提供了几种类型的分支语句

- 单分支if语句
- 双分支if-else语句
- 嵌套if语句
- switch语句
- 条件表达式

单分支if语句语法:

```
if (bool expression)
{
    statement(s);
}
```

布尔表达式必须在括号内

可以没有语句、仅1个语句或多个语句

只有1个语句时，花括号可以省略

```
#include <iostream>
using namespace std;
```

```
int main()
{
    // Prompt the user to enter an integer
    int number;
    cout << "Enter an integer: ";
    cin >> number;
```

```
    if (number % 5 == 0)
        cout << "HiFive" << endl;
```

```
    if (number % 2 == 0)
        cout << "HiEven" << endl;
```

```
    return 0;
}
```

D:\Edu-CPP\chap03\ex03-01.exe

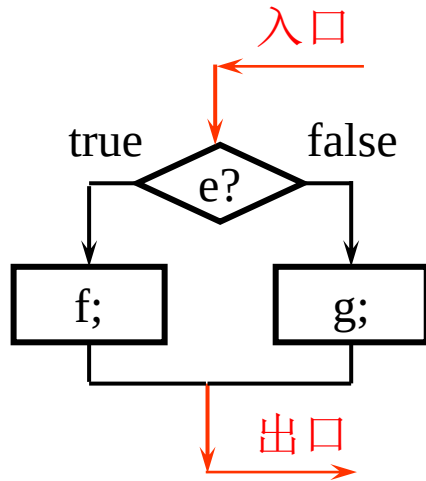
```
Enter an integer: 20
HiFive
HiEven
```




双分支if-else语句

◆ **if-else**语句根据布尔表达式值的真假，决定应该执行哪条语句

```
if(booleanExpression)
{
    statement(s) f;
}
else
{
    statement(s) g;
}
```



```
#include <iostream>
using namespace std;
int main()
{
    double radius, area;
    const double PI = 3.14159;

    cout << "Enter a radius: ";
    cin >> radius;
    if (radius < 0)
        cout << "Incorrect Input" << endl;
    else
    {
        area = radius * radius * PI;
        cout << "The area is " << area << endl;
    }
    return 0;
}
```

嵌套的if语句和多分支的if-else语句



◆ 嵌套的if语句

```
if (bool expression)
{
    statement(s);
}
```

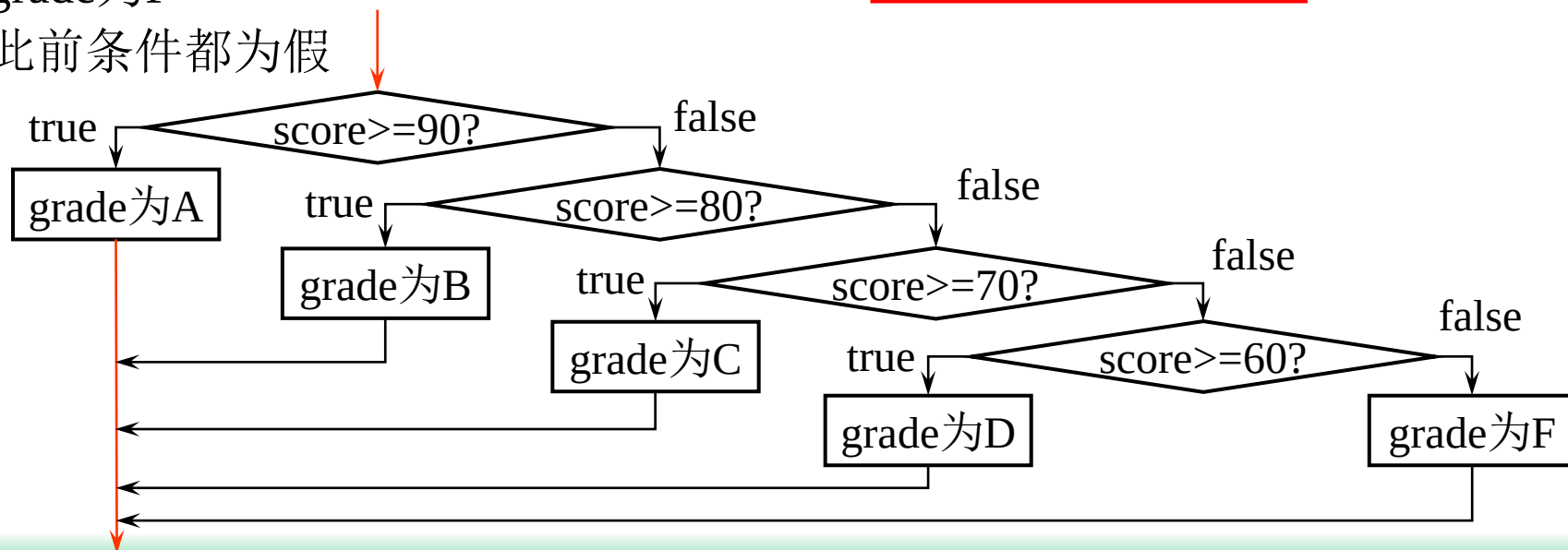
可以是if语句；是if语句时，则构成if语句嵌套

```
if(booleanExpression)
{
    statement(s) f;
}
else
{
    statement(s) g;
}
```

■ 嵌套的if语句可以用于实现多个选择的情况

- 全部条件都为假时grade为F
- 后续判断的前提是此前条件都为假

将百分制转换为五级制的流程图

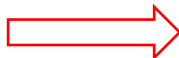




◆ 某些时候，将嵌套的if语句改写为多分支if-else语句，代码风格更好，避免过深的if语句嵌套缩进

```
if(score >= 90.0)
    cout << "Grade is A";
else
    if(score >= 80.0)
        cout << "Grade is B";
    else
        if(score >= 70.0)
            cout << "Grade is C";
        else
            if(score >= 60.0)
                cout << "Grade is D";
            else
                cout << "Grade is F";
```

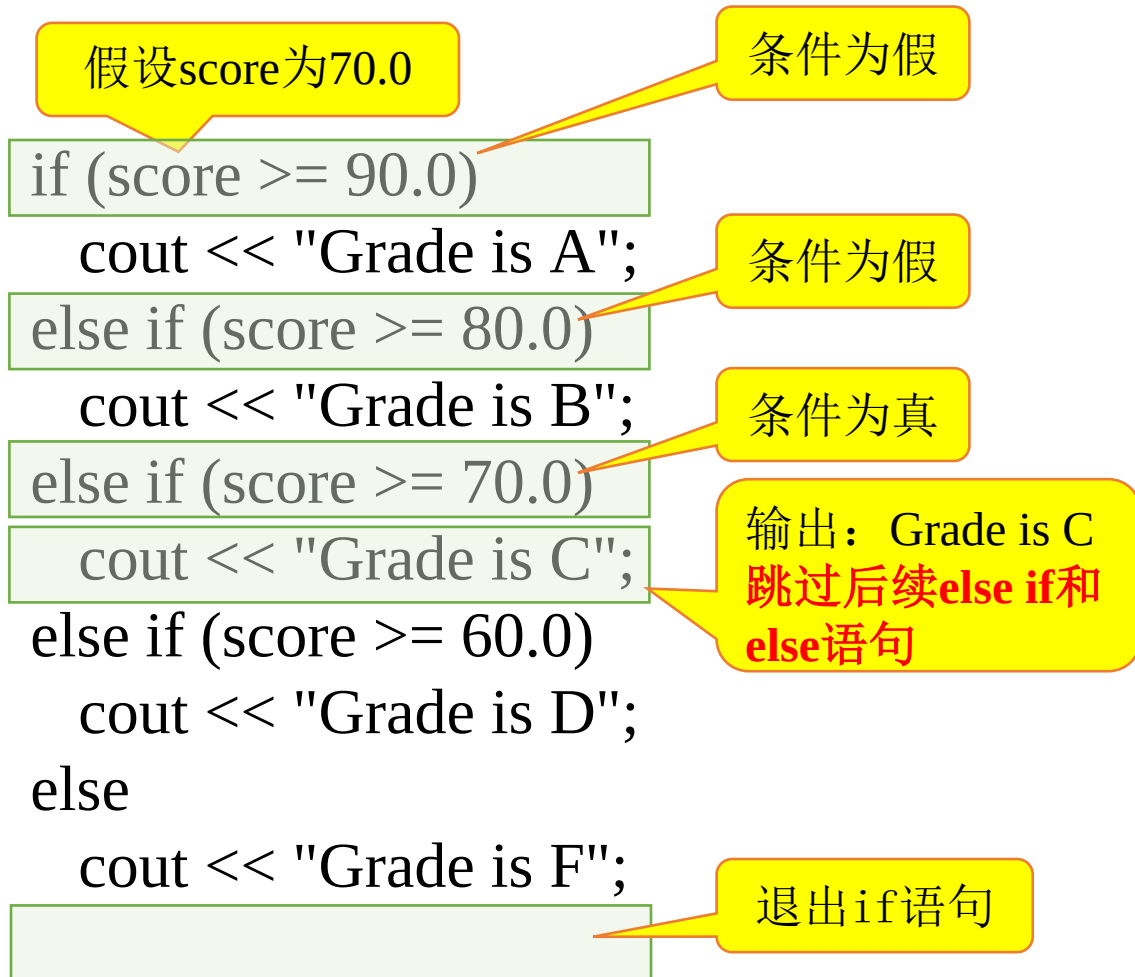
等价于



```
if(score >= 90.0)
    cout << "Grade is A";
else if(score >= 80.0)
    cout << "Grade is B";
else if(score >= 70.0)
    cout << "Grade is C";
else if(score >= 60.0)
    cout << "Grade is D";
else
    cout << "Grade is F";
```



◆跟踪if-else语句，理解其执行流程



```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      float score = 70.0f;
6
7      if(score >= 90.0)
8          cout << "Grade is A";
9      else if(score >= 80.0)
10         cout << "Grade is B";
11     else if(score >= 70.0)
12         cout << "Grade is C";
13     else if(score >= 60.0)
14         cout << "Grade is D";
15     else
16         cout << "Grade is F";
17
18     return 0;
19 }
```



◆ 下面的代码为什么不正确？

```
if(score >= 60.0)
    cout << "Grade is D";
else if(score >= 70.0)
    cout << "Grade is C";
else if(score >= 80.0)
    cout << "Grade is B";
else if(score >= 90.0)
    cout << "Grade is A";
else
    cout << "Grade is F";
```



◆ 假设 $x=3$, $y=2$, 给出下列代码的输出结果; 若 $x=3$, $y=4$, 输出是什么? 若 $x=2$, $y=2$, 输出是什么? 请为代码绘制流程图

```
if(x > 2)
{
    if(y > 2)
    {
        int z = x + y;
        cout << "z is " << z << endl;
    }
}
else
    cout << "x is " << x << endl;
```

```
> D:\Edu-CPP\chap03\ex03-02.exe
```

```
Enter x and y: 3 2
```

```
> D:\Edu-CPP\chap03\ex03-02.exe
```

```
Enter x and y: 3 4
z is 7
```

```
> D:\Edu-CPP\chap03\ex03-02.exe
```

```
Enter x and y: 2 2
x is 2
```

本章主要内容



3.1 **bool**（布尔）数据类型、关系运算符与布尔表达式

3.2 **if**语句和**if-else**语句

3.3 **if**语句常见错误和陷阱



3.4 逻辑运算符

3.5 **switch**语句

3.6 条件表达式

3.7 运算符优先级和结合律

3.8 作业与实践

常见错误



◆ 常见错误1：忘记必须的花括号

```
if (radius < 0)
    cout << "Incorrect Input" << endl;
else
    area = radius * radius * PI;
    cout << "The area is " << area << endl;
```

不在if-else的“管理范围内”，始终被执行

```
if (radius < 0)
    cout << "Incorrect Input" << endl;
else
{
    area = radius * radius * PI;
    cout << "The area is " << area << endl;
}
```

if及else语句后面有效范围都只有一个语句，如果包含多个语句时，要用花括号“{}”将这些语句括起来成为一个复合语句

◆ 常见错误2：if行错误的分号

■ 逻辑错误

```
if(y > 2);
{
    int z = x + y;
    cout << "z is " << z << endl;
}
```

等价于

```
if(y > 2) { };
{
    int z = x + y;
    cout << "z is " << z << endl;
}
```

花括号代码段不受if语句控制，始终被执行



◆ 常见错误3：错误使用=代替==

```
if (count = 0)
    cout << "count is zero" << endl;
else
    cout << "count is not zero" <<
endl;
```

始终不执行

始终不执行

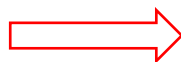
```
if (count = 1)
    cout << "count is one" << endl;
else
    cout << "count is not one" <<
endl;
```

◆ 常见错误4：布尔值的冗余判断

■ 避免发生错误3之类的错误

```
if(even == true)
    cout << "It is even.";
```

优化为



```
if(even)
    cout << "It is even.";
```



◆ 常见错误5: else位置歧义

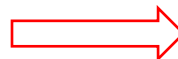
■ if可独立使用, else则必须和if配对

■ else与if匹配原则: else与其之前距离最近且没有与其它else配对的if 配对

➤ 不是根据排版退缩

```
int i = 1, j = 2, k = 3;  
if(i > j)  
    if(i > k)  
        cout << "A";  
else  
    cout << "B";
```

等价于



```
int i = 1, j = 2, k = 3;  
if(i > j)  
    if(i > k)  
        cout << "A";  
else  
    cout << "B";
```

没有任何输出

```
int i = 1, j = 2, k = 3;  
if(i > j)  
{  
    if(i > k)  
        cout << "A";  
}  
else  
    cout << "B";
```

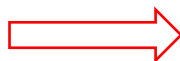
代码排版意图的正确表达



◆ 常见错误6：两个浮点数的相等性判断

```
double x = 1 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1;  
if(x == 0.5)  
    cout << "x is 0.5" << endl;  
else  
    cout << "x is not 0.5" << endl;
```

更正



```
double x = 1 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1;  
if(abs(x - 0.5) < 1E-14)  
    cout << "x is 0.5" << endl;  
else  
    cout << "x is not 0.5" << endl;
```

<cmath>中的函数abs()
返回数的绝对值
double: 1E-14
float: 1E-7

本例始终执行else，某些情况可能执行为真时的语句，由具体浮点数的实际精度误差决定

D:\Edu-CPP\chap03\ex03-03.exe

x is not 0.5

D:\Edu-CPP\chap03\ex03-03B.exe

x is 0.5

```
double x = 0.500000000000000011 - 0.1 - 0.1;  
if(x == 0.5)  
    cout << "x is 0.5" << endl;  
else  
    cout << "x is not 0.5" << endl;
```



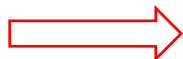
常见陷阱

◆ 陷阱**不一定是错误**，但让代码不简洁、阅读代码易误解、代码不易维护等

◆ 常见陷阱1：简化布尔变量赋值

```
if (number % 2 == 0)
    even = true;
else
    even = false;
```

优化为



```
bool even = number % 2 == 0;
```

【1】用布尔表达式重写，更简洁；
【2】涉及运算符的优先级，后面讲解；
【3】可以将赋值符及布尔表达式放在第2行：
bool even
 = number % 2 == 0;

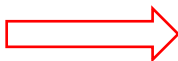


◆ 常见陷阱2：避免在不同分支中的相同语句

■ 以便维护代码（避免修改代码须多处修改）

```
if(inState)
{
    tuition = 5000;
    cout << "The tuition is " << tuition << endl;
}
else
{
    tuition = 15000;
    cout << "The tuition is " << tuition << endl;
}
```

优化为



```
if(inState)
    tuition = 5000;
else
    tuition = 15000;
cout << "The tuition is " << tuition << endl;
```



◆ 常见陷阱3：整数值可以被用作布尔值

■ 非0为true，0为false

```
int amount = 40;  
if(!amount <= 50)  
    cout << "Amount is more than 50";
```

更正为



```
int amount = 40;  
if(!(amount <= 50))  
    cout << "Amount is more than 50";
```

- 【1】 “!” 的优先级高于 “<=” ；
- 【2】 此处 “!amount” 等价于对true取反，即等价于0；
- 【3】 属于逻辑错误，本意：amount并不小于等于50时，显示 “amount大于50”

```
> D:\Edu-CPP\chap03\ex03-03C.exe
```

```
Amount is more than 50
```

示例：计算身体质量指数



◆ 身体质量指数（BMI）是一种利用体重和身高衡量健康的方法，体重的千克数除以身高米数的平方即为BMI。大于20岁人的BMI指数解释如表所示

BMI	解释
$BMI < 18.5$	偏瘦
$18.6 \leq BMI < 25.0$	正常
$25.0 \leq BMI < 30.0$	超重
$30.0 \leq BMI$	肥胖

■ 编写程序提示用户输入以磅为单位的体重和以英尺为单位的身高，然后显示BMI。1磅是0.45359237千克，1英寸是0.0254米

```
#include <iostream>
using namespace std;
```

```
int main()
{
```

```
    // Prompt the user to enter weight in pounds
```

```
    cout << "Enter weight in pounds: ";
```

```
    double weight;
```

```
    cin >> weight;
```

```
    // Prompt the user to enter height in inches
```

```
    cout << "Enter height in inches: ";
```

```
    double height;
```

```
    cin >> height;
```

```
    const double KILOGRAMS_PER_POUND = 0.45359237; // Constant
```

```
    const double METERS_PER_INCH = 0.0254; // Constant
```

```
    // Compute BMI
```

```
    double weightInKilograms = weight * KILOGRAMS_PER_POUND;
```

```
    double heightInMeters = height * METERS_PER_INCH;
```

```
    double bmi = weightInKilograms / (heightInMeters * heightInMeters);
```

此时的条件是: $18.6 \leq \text{BMI} < 25.0$,
为什么?

```
// Display result
```

```
cout << "BMI is " << bmi << endl;
```

```
if (bmi < 18.5)
```

```
    cout << "Underweight" << endl;
```

```
else if (bmi < 25)
```

```
    cout << "Normal" << endl;
```

```
else if (bmi < 30)
```

```
    cout << "Overweight" << endl;
```

```
else
```

```
    cout << "Obese" << endl;
```

```
    return 0;
```

```
}
```

D:\Edu-CPP\chap03\ex03-04.exe

```
Enter weight in pounds: 146
Enter height in inches: 70
BMI is 20.9486
Normal
```


示例：基于随机数的减法练习程序



- ◆ 编写程序随机生成两个一位整数`number1`和`number2`，且`number1 ≥ number2`（假设这两个数分别是9和2），提示学生回答“**What is 9 – 2?**”，学生输入答案后程序显示信息说明答案是否正确
 - 步骤1：生成两个一位的整数`number1`和`number2`
 - 利用`<cstdlib>`头文件中的**`rand()`**函数生成**随机整数**
 - `rand()`生成的是**伪随机数**，即在同一系统中每次生成**同一序列**的数，须用不同种子生成不同序列的数，可以用`time(0)`产生不同种子`seed`，用**`srand(seed)`**函数改变种子的值
 - 利用`rand() % 10`获取**0~9**的随机整数
 - 步骤2：若`number1 < number2`，则交换`number1`和`number2`的值
 - **交换算法：**`tmp = number1; number1 = number2; number2 = tmp;`
 - 步骤3：提示学生回答`number1 – number2`的结果是多少
 - 步骤4：检查学生的回答是否正确，并显示是否正确的相应信息

```

#include <iostream>
#include <ctime> // for time( )
#include <cstdlib> // for rand( ) and srand( )
using namespace std;

int main()
{
    // 1. Generate two random single-digit integers
    srand(time(0));
    int number1 = rand() % 10;
    int number2 = rand() % 10;

    // 2. If number1 < number2, swap them
    if (number1 < number2)
    {
        int temp = number1;
        number1 = number2;
        number2 = temp;
    }
}

```

注释掉这一行语句，结果会如何？
多次运行程序试试

- 【1】 如何获取0~999的随机整数？
- 【2】 如何获取30~55的随机整数？

```

// 3. Prompt the student to answer "What is number1 - number2?"
cout << "What is " << number1 << " - " << number2 << "? ";
int answer;
cin >> answer;

// 4. Grade the answer and display the result
if (number1 - number2 == answer)
    cout << "You are correct!";
else
    cout << "Your answer is wrong." << endl << number1 << " - "
    << number2 << " should be " << (number1 - number2) << endl;

return 0;
}

```

D:\Edu-CPP\chap03\ex03-05.exe

```

What is 9 - 6? 3
You are correct!

```

D:\Edu-CPP\chap03\ex03-05.exe

```

What is 4 - 2? 1
Your answer is wrong.
4 - 2 should be 2

```

本章主要内容



3.1 **bool**（布尔）数据类型、关系运算符与布尔表达式

3.2 **if**语句和**if-else**语句

3.3 **if**语句常见错误和陷阱

3.4 逻辑运算符



3.5 **switch**语句

3.6 条件表达式

3.7 运算符优先级和结合律

3.8 作业与实践

逻辑运算符及其定义



◆ **逻辑运算符**也称作**布尔运算符**，是以布尔值为运算对象，计算出新的布尔值的运算符，有三种逻辑运算符，如表所示

运算符	名字	举例
&&	逻辑与	$a > 2 \ \&\& \ a < 3$
	逻辑或	$s < 2 \ \ s > 6$
!	逻辑非	$!a$

在数学中，表达式 $1 \leq x < 9$ 是正确的，但在C/C++中是一种**逻辑错误**：

$1 \leq x$ 的值是布尔值1或0，该布尔值恒小于9，即该关系表达式的值恒为真，正确写法应是 $1 \leq x \ \&\& \ x < 9$

&&的各种操作（真值表）

- $\text{true} \ \&\& \ \text{true} = \text{true}$
- $\text{true} \ \&\& \ \text{false} = \text{false}$
- $\text{false} \ \&\& \ \text{false} = \text{false}$

两操作数都为true，结果为true，否则为false

||的各种操作（真值表）

- $\text{true} \ || \ \text{true} = \text{true}$
- $\text{true} \ || \ \text{false} = \text{true}$
- $\text{false} \ || \ \text{false} = \text{false}$

两操作数中至少一个为true，结果为true，否则为false

!的各种操作（真值表）

- $!\text{true} = \text{false}$
- $!\text{false} = \text{true}$

操作数为true，结果为false，否则为true

示例：检查整除情况



- ◆ 编程检查一个整数是否被2和3同时整除，是否被2或被3整除，是否被且只被2、3其中之一整除

```
#include <iostream>
using namespace std;
int main() {
    int number;
    cout << "Enter an integer: ";
    cin >> number;

    if (number % 2 == 0 && number % 3 == 0)
        cout << number << " is divisible by 2 and 3." << endl;
    if (number % 2 == 0 || number % 3 == 0)
        cout << number << " is divisible by 2 or 3." << endl;
    if ((number % 2 == 0 || number % 3 == 0) &&
        !(number % 2 == 0 && number % 3 == 0))
        cout << number << " divisible by 2 or 3, but not both." << endl;

    return(0);
}
```

D:\Edu-CPP\chap03\ex03-06.exe

```
Enter an integer: 4
4 is divisible by 2 or 3.
4 divisible by 2 or 3, but not both.
```

D:\Edu-CPP\chap03\ex03-06.exe

```
Enter an integer: 18
18 is divisible by 2 and 3.
18 is divisible by 2 or 3.
```

布尔表达式的等价变换与简化



◆ De Morgan定律

`!(condition1 && condition2)` 等同于
`!condition1 || !condition2`

`!(condition1 || condition2)` 等同于
`!condition1 && !condition2`

■ 例如:

`!(number % 2 == 0 && number % 3 == 0)` 可简化为等价表达式:
`number % 2 != 0 || number % 3 != 0`

`!(number == 2 || number == 3)` 优化为等价表达式:
`number != 2 && number != 3`

◆ 若运算符`&&`的一个操作数为`false`, 则表达式值为`false`; 若运算符`||`的一个操作数为`true`, 表达式值为`true`。C/C++用此“**短路特性**”优化:

■ 求`p1 && p2`的值时, 先求`p1`的值, 若`p1`为`true`则求`p2`的值, 否则不求`p2`的值

■ 求`p1 || p2`的值时, 先求`p1`的值, 若`p1`为`false`则求`p2`的值, 否则不求`p2`的值

示例：判断某年是否是闰年



- ◆ 闰年是可以被4整除，但不能被100整除的年份，或者是能被400整除的年份

```
#include <iostream>
using namespace std;
int main() {
    cout << "Enter a year: ";
    int year;
    cin >> year;
    // Check if the year is a leap year
    bool isLeapYear = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    // Display the result in a message dialog box
    if (isLeapYear)
        cout << year << " is a leap year" << endl;
    else
        cout << year << " is a not leap year" << endl;
    return 0;
}
```

D:\Edu-CPP\chap03\ex03-07.exe

Enter a year: 2008
2008 is a leap year

D:\Edu-CPP\chap03\ex03-07.exe

Enter a year: 1900
1900 is a not leap year

示例：用逻辑运算符改写嵌套的if语句



```
if(score >= 90.0)
    cout << "你最棒！" << endl;
else if(score >= 80.0)
    cout << "你很优秀！" << endl;
else if(score >= 70.0)
    cout << "你很不错！" << endl;
else if(score >= 60.0)
    cout << "你及格了！" << endl;
else if(score >= 50.0)
    cout << "你离成功不远啦！" << endl;
else
    cout << "你丫的根本没学习！" << endl;
```

```
if(score >= 90.0)
    cout << "你最棒！" << endl;
if(score < 90.0 && score >= 80.0)
    cout << "你很优秀！" << endl;
if(score < 80.0 && score >= 70.0)
    cout << "你很不错！" << endl;
if(score < 70.0 && score >= 60.0)
    cout << "你及格了！" << endl;
if(score < 60.0 && score >= 50.0)
    cout << "你离成功不远啦！" << endl;
if(score < 50.0)
    cout << "你丫的根本没学习！" << endl;
```

功能相同，但执行效率
相对较低，为什么？

本章主要内容



3.1 bool（布尔）数据类型、关系运算符与布尔表达式

3.2 if语句和if-else语句

3.3 if语句常见错误和陷阱

3.4 逻辑运算符

3.5 switch语句



3.6 条件表达式

3.7 运算符优先级和结合律

3.8 作业与实践

switch语句的语法



switch (表达式)

{

case 常量表达式1: 语句序列1; **break**;

case 常量表达式2: 语句序列2; **break**;

.....

.....

case 常量表达式n: 语句序列n; **break**;

default: 语句序列n+1;

}

多个case可共用
一组执行语句

先求表达式的值 (**整型**)

依次比较表达式值和各常量表达式值 (**整型**)，若和第i个常量表达式相等，则执从该case语句执行后续语句，直至遇到一个break语句或到达switch语句末尾的花括号

若不相等，则执行default的语句序列n+1

default情况是可选的，给出任何情况 (case) 都与switch表达式不匹配时执行的语句

break语句是可选的 (可有可无)，break语句将立即终止switch语句

示例：用switch语句改写嵌套的if语句



```
if(score >= 90.0)
    cout << "你最棒！" << endl;
else if(score >= 80.0)
    cout << "你很优秀！" << endl;
else if(score >= 70.0)
    cout << "你很不错！" << endl;
else if(score >= 60.0)
    cout << "你及格了！" << endl;
else if(score >= 50.0)
    cout << "你离成功不远啦！" << endl;
else
    cout << "你丫的根本没学习！" << endl;
```

≥ 90	"你最棒！"
80~89	"你很优秀！"
70~79	"你很不错！"
60~69	"你及格了！"
50~59	"你离成功不远啦！"
≤ 49	"你丫的根本没学习！"

```

#include <iostream>
using namespace std;
int main() {
    int score;
    cout << "Enter score: ";    cin >> score;
    switch(score/10) {
        case 10:
        case 9: cout << "你最棒！" << endl; break ;
        case 8: cout << "你很优秀！" << endl; break;
        case 7: cout << "你很不错！" << endl; break;
        case 6: cout << "你及格了！" << endl; break;
        case 5: cout << "你离成功不远啦！" << endl; break;
        case 4:
        case 3:
        case 2:
        case 1:
        case 0: cout << "你丫的根本没学习！" << endl; break;
        default: cout << "输入数据不合理\n"; break;
    }
    return 0;
}

```

case
后面
整型
常量
互不
相同

整型或字符型表达式

多个case可共用
一组执行语句

注释掉该break语句，输
入分数为66，输出是？

D:\Edu-CPP\chap03\ex03-08.exe

Enter score: 100
你最棒！

D:\Edu-CPP\chap03\ex03-08.exe

Enter score: 20
你丫的根本没学习！

D:\Edu-CPP\chap03\ex03-08.exe

Enter score: -10
输入数据不合理

D:\Edu-CPP\chap03\ex03-08.exe

Enter score: 66
你及格了！
你离成功不远啦！

本章主要内容



3.1 **bool**（布尔）数据类型、关系运算符与布尔表达式

3.2 **if**语句和**if-else**语句

3.3 **if**语句常见错误和陷阱

3.4 逻辑运算符

3.5 **switch**语句

3.6 条件表达式



3.7 运算符优先级和结合律

3.8 作业与实践



◆ 有时候需要根据条件给一个变量赋不同的值，例如：

```
if (x >= 0)
    y = 1;
else
    y = -1;
```

等价于采用条件表达式

```
y = x >= 0 ? 1 : -1;
```

◆ 条件表达式

■ 语法：e1 ? e2 : e3

➤ 问号?和冒号:一起构成**条件运算符**（三元运算符）

■ 运算

➤ 首先对表达式e1求值

➤ 若e1值为真，则计算e2作为条件表达式的值

➤ 若e1值为假，则计算e3作为条件表达式的值

```
int a=4, b=9, max;
max = (a>b) ? a : b;
```

本章主要内容



3.1 **bool**（布尔）数据类型、关系运算符与布尔表达式

3.2 **if**语句和**if-else**语句

3.3 **if**语句常见错误和陷阱

3.4 逻辑运算符

3.5 **switch**语句

3.6 条件表达式

3.7 运算符优先级和结合律



3.8 作业与实践

优先级

第一原则：单目运算
优先级高于双目运算

高	!、++、--	优先级相同
	*、/、%	优先级相同
	+、-	优先级相同
	>、>=、<、<=	优先级相同
	==、!=	优先级相同
	&&	
	? :	
低	=、+=、-=、*=、/=、%=	

第二原则：
算术运算

关系运算

逻辑运算

条件运算

赋值运算

结合性

右→左

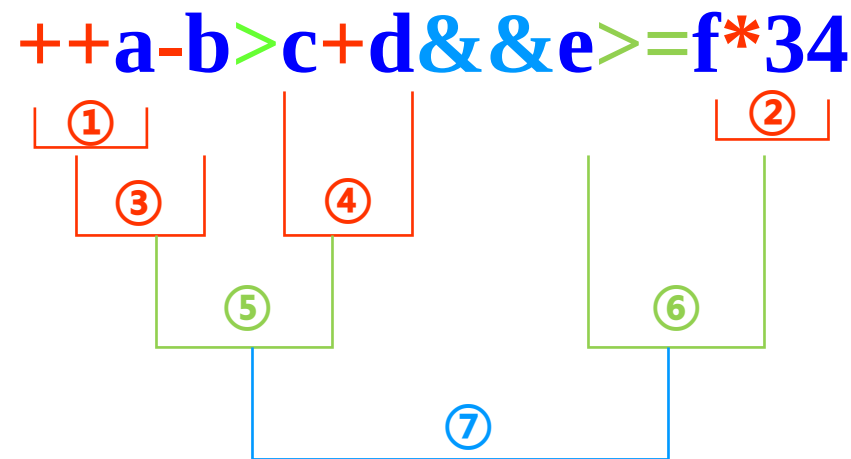
左→右
左→右

左→右
左→右

左→右
左→右

右→左

右→左



按照优先级顺序执行，具有相同
优先级则按照结合性控制执行

() 为最高优先级，如果代码行
中的运算符比较多，用括号确定
表达式的操作顺序

运算符优先级与结合性示例



$a = b = c$	等价于	$a = (b = c)$	右结合
$a + b + c$	等价于	$(a + b) + c$	左结合
$3 + 4 + 5 - 8$	等价于	$((3 + 4) + 5) - 8$	
$3 * 4 / 5 * 8$	等价于	$((3 * 4) / 5) * 8$	
$3 * 4 + 8 / 5$	等价于	$(3 * 4) + (8 / 5)$	

给出下列表达式的值：

true || true && false

true && true || false

$2 * 2 - 3 > 2 \&\& 4 - 2 >= 5$

$2 * 2 - 3 > 2 \parallel 4 - 2 >= 5$

$a = b += c = 5$ 等价于 $a = (b += (c = 5))$

赋值运算符是**右结合**

a、b、c初值均为1，则整个表达式执行完之后，
c为5，b为6，a为6

本章主要内容



3.1 **bool**（布尔）数据类型、关系运算符与布尔表达式

3.2 **if**语句和**if-else**语句

3.3 **if**语句常见错误和陷阱

3.4 逻辑运算符

3.5 **switch**语句

3.6 条件表达式

3.7 运算符优先级和结合律

3.8 作业与实践



第03章作业



(1)求二次方程 $ax^2 + bx + c=0$ 的根。编写程序提示用户输入a、b、c的值，输出基于判别式的结果。若判别式为正，输出两个根；若判别式为0，输出一个根；若判别式为负，输出“无实根”

■可以用 $\text{pow}(x, 0.5)$ 求 $\sqrt{x}$

(2)编写程序提示用户输入温度，若温度低于20，输入“太冷”；若温度高于30度，输出“太热”；否则，输出“正好”

(3)编写程序提示用户输入代表今天是星期几的整数（星期日为0，星期一为1，……），然后提示用户输入未来某天距离今天共几天，输出未来这天为星期几

(4)编程提示用户输入3个整数，输出这3个整数的升序排序结果

(5)编写程序提示用户输入月份和年份，输出该月的天数

■注意闰年



(6)运输公司用下面分段函数根据包裹重量（以磅为单位）计算运输费用（以美元为单位）。编写程序提示用户输入包裹重量，输出运费，如果包裹重量超过20，输出“超重”

$$c(w) = \begin{cases} 3.5, & 0 < w \leq 1 \\ 5.5, & 1 < w \leq 3 \\ 8.5, & 3 < w \leq 10 \\ 10.5, & 10 < w \leq 20 \end{cases}$$

(7)编写程序读入一个三角形的三边长度，若输入合法，计算并输出该三角形的周长，否则提示输入非法

(8)编写程序提示用户输入点 (x, y) ，检查该点是否在以 $(0, 0)$ 为圆心，半径为10的圆内

(9)编写程序提示用户输入一个三位整数，判断该数是否是回文数。若一个数从右往左与从左往右读是一样的，则该数是回文数，如121